



10-X23 Generative AI

Machine Learning Department
School of Computer Science
Carnegie Mellon University

Learning Large Language Models

Matt Gormley
Lecture 3

Reminders

- **Homework 0: PyTorch + Weights & Biases**
 - Out: Wed, Jan 14
 - Due: Mon, Jan 26 at 11:59pm
- **Homework 1: Generative Models of Text**
 - Out: Mon, Jan 26
 - Due: Mon, Feb 9 at 11:59pm

Q&A

Q: How will I earn the 5% participation points?

A: Very gradually. There will be a few aspects of the course (polls, surveys, meetings with the course staff) that we will attach participation points to.

RECAP

Module-based AutoDiff (oOP Version)

Object-Oriented Implementation:

- Let each module be an **object**
- Then allow the **control flow** dictate the creation of the **computation graph**
- No longer need to implement NNBackward($\cdot$), just follow the computation graph in **reverse topological order**

```

1 class Sigmoid(Module)
2     method forward(a)
3          $b = \sigma(a)$ 
4         return b
5     method backward(a, b, gb)
6          $g_a = g_b \odot b \odot (1 - b)$ 
7         return ga

```

```

1 class Softmax(Module)
2     method forward(a)
3          $b = \text{softmax}(a)$ 
4         return b
5     method backward(a, b, gb)
6          $g_a = g_b^T (\text{diag}(b) - bb^T)$ 
7         return ga

```

```

1 class Linear(Module)
2     method forward(a,  $\omega$ )
3          $b = \omega a$ 
4         return b
5     method backward(a,  $\omega$ , b, gb)
6          $g_\omega = g_b a^T$ 
7          $g_a = \omega^T g_b$ 
8         return g $_\omega$ , ga

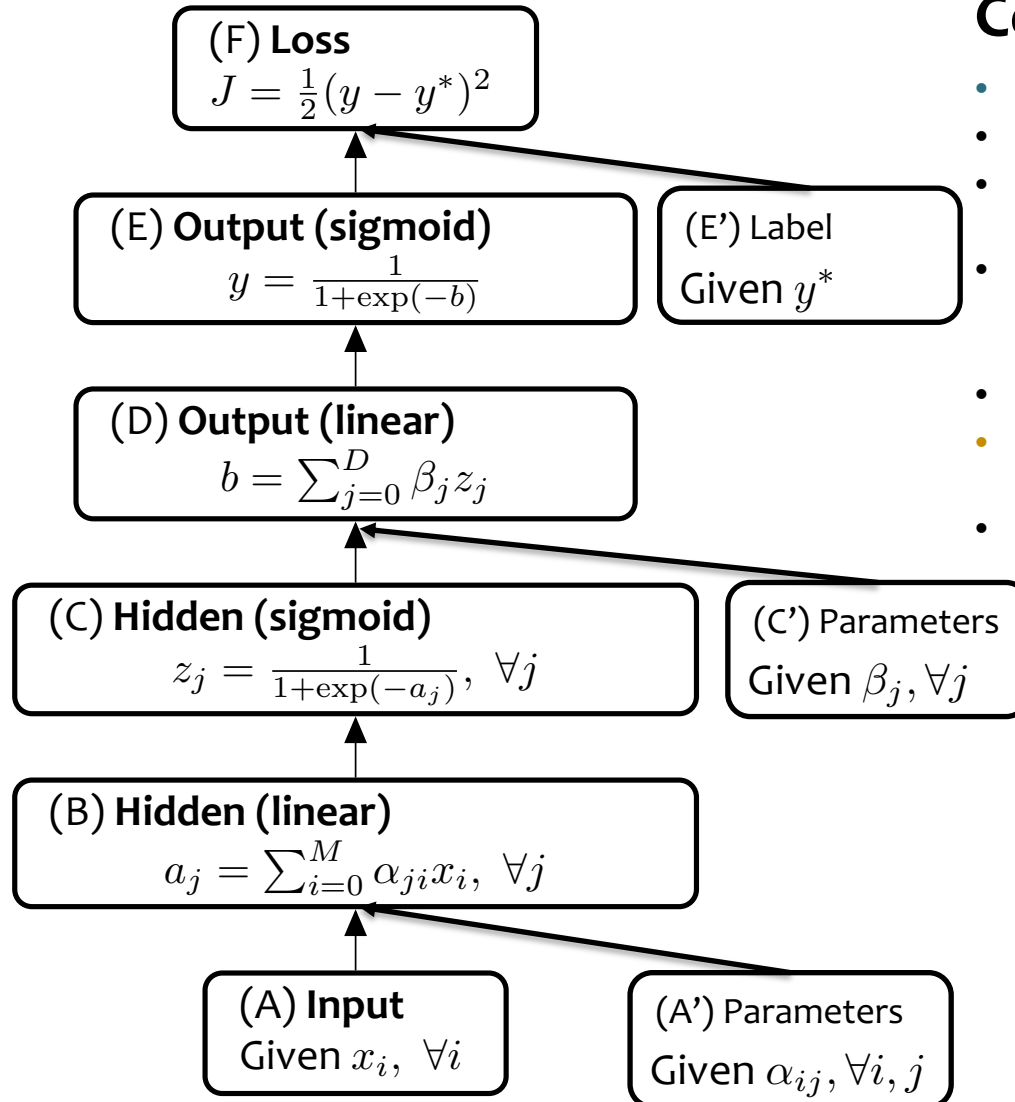
```

```

1 class CrossEntropy(Module)
2     method forward(a,  $\hat{a}$ )
3          $b = -a^T \log \hat{a}$ 
4         return b
5     method backward(a,  $\hat{a}$ , b, gb)
6          $g_{\hat{a}} = -g_b (a \div \hat{a})$ 
7         return ga

```

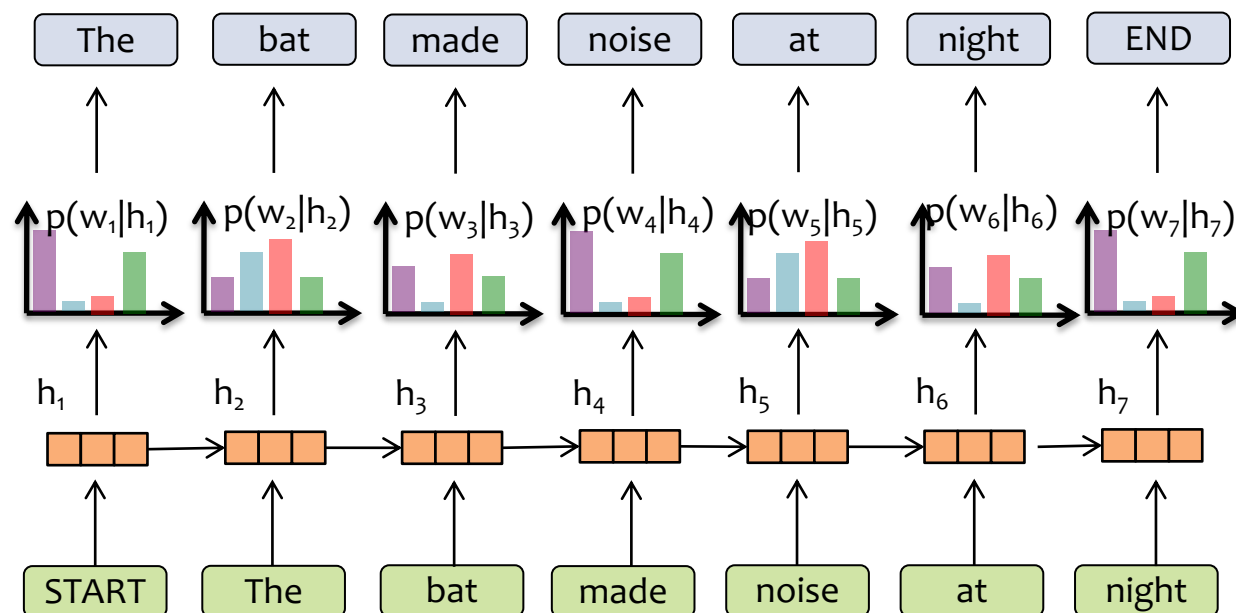
Ways of Drawing Neural Networks



Computation Graph

- The diagram represents an algorithm
- Nodes are **rectangles**
- One node per **intermediate variable in the algorithm**
- Node is labeled with the **function** that it computes (inside the box) and also the **variable** name (outside the box)
- Edges are directed
- Edges do not have labels (since they don't need them)
- For neural networks:
 - Each **intercept term** should appear as a node (if it's not folded in somewhere)
 - Each parameter should appear as a node
 - Each constant, e.g. a true label or a feature vector should appear in the graph
 - It's **perfectly fine** to include the loss

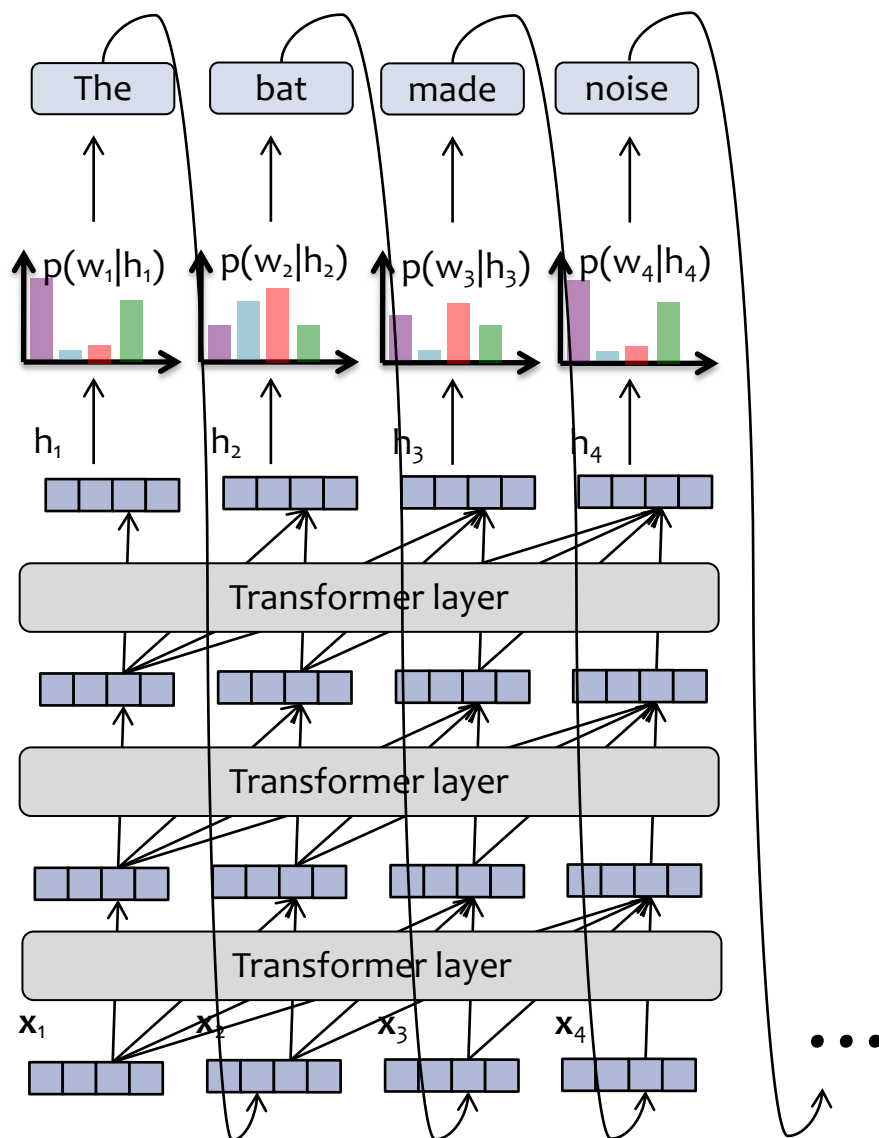
RNN Language Model



Key Idea:

- (1) convert all previous words to a **fixed length vector**
- (2) define distribution $p(w_t | f_{\theta}(w_{t-1}, \dots, w_1))$ that conditions on the vector $\mathbf{h}_t = f_{\theta}(w_{t-1}, \dots, w_1)$

Transformer Language Model



Each layer of a Transformer LM consists of several **sublayers**:

1. attention
2. feed-forward neural network
3. layer normalization
4. residual connections

Each hidden vector looks back at the hidden vectors of the **current and previous timesteps in the previous layer**.

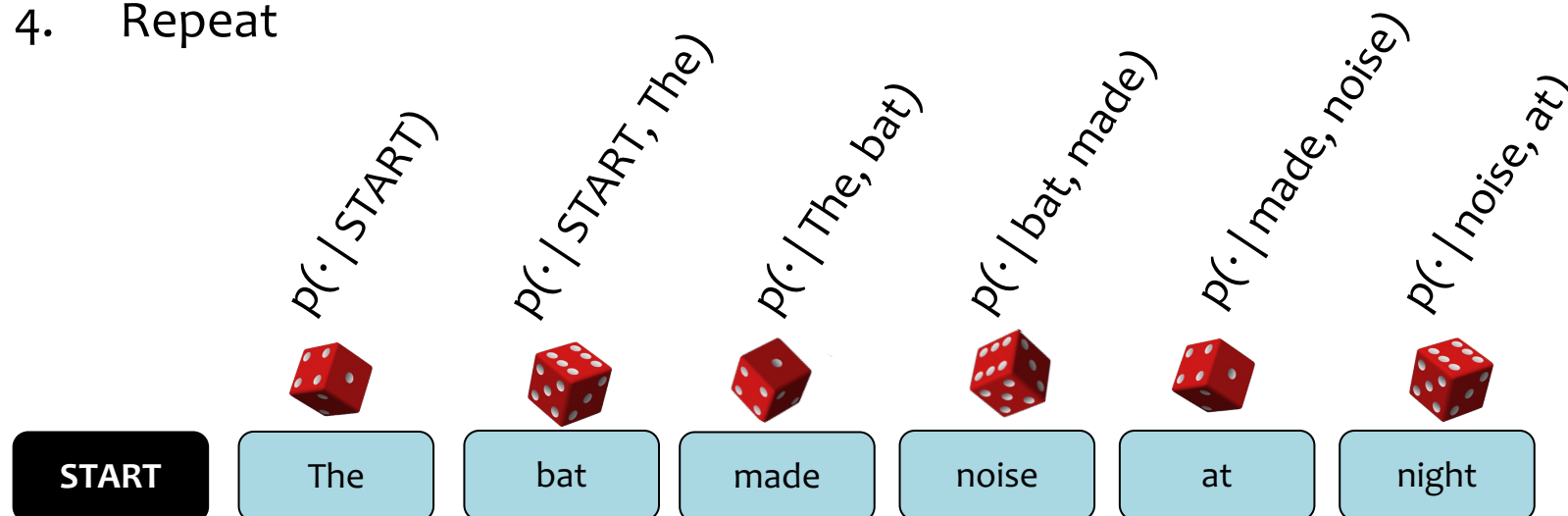
The language model part is just like an RNN-LM.

Sampling from a Language Model

Question: How do we sample from a Language Model?

Answer:

1. Treat each probability distribution like a (50k-sided) weighted die
2. Pick the die corresponding to $p(w_t | w_{t-2}, w_{t-1})$
3. Roll that die and generate whichever word w_t lands face up
4. Repeat



Recap So Far

Deep Learning

- AutoDiff
 - is a tool for **computing gradients** of a differentiable function, $b = f(a)$
 - the key building block is a **module** with a `forward()` and `backward()`
 - sometimes define f as **code** in `forward()` by chaining existing modules together
- Computation Graphs
 - are another way to define f (more conducive to slides)
 - so far, we saw two (deep) computation graphs
 - 1) RNN-LM
 - 2) Transformer-LM
 - (Transformer-LM was kind of complicated)

Language Modeling

- key idea: condition on previous words to **sample the next word**
- to define the **probability** of the next word...
 - ...n-gram LM uses collection of massive 50k-sided **dice**
 - ...RNN-LM or Transformer-LM use a **neural network**
- Learning an LM
 - n-gram LMs are easy to learn: just **count** co-occurrences!
 - so far, we said **nothing about how to learn** an RNN-LM or Transformer-LM
 - So let's figure that out next...

LEARNING A NEURAL NETWORK

A Recipe for Machine Learning

Recall...

1. Given training data:

$$\{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^N$$

2. Choose each of these:

- Decision function

$$\hat{\mathbf{y}} = f_{\boldsymbol{\theta}}(\mathbf{x}_i)$$

- Loss function

$$\ell(\hat{\mathbf{y}}, \mathbf{y}_i) \in \mathbb{R}$$

3. Define goal:

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} \sum_{i=1}^N \ell(f_{\boldsymbol{\theta}}(\mathbf{x}_i), \mathbf{y}_i)$$

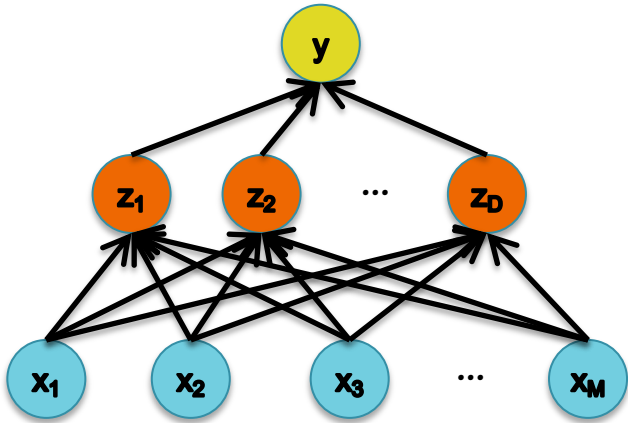
4. Train with SGD:

(take small steps
opposite the gradient)

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta_t \nabla \ell(f_{\boldsymbol{\theta}}(\mathbf{x}_i), \mathbf{y}_i)$$

Backpropagation

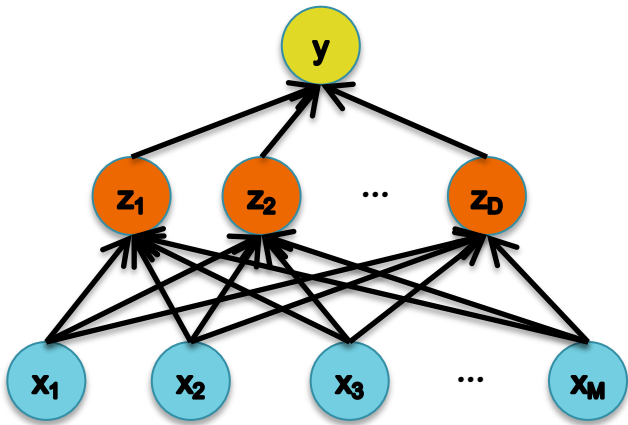
**Example:
Neural Network**



	Forward	Backward
Loss	$J = y^* \log y + (1 - y^*) \log(1 - y)$	$g_y = \frac{y^*}{y} + \frac{(1 - y^*)}{y - 1}$
Sigmoid	$y = \frac{1}{1 + \exp(-b)}$	$g_b = g_y \frac{\partial y}{\partial b}, \frac{\partial y}{\partial b} = y(1 - y)$
Linear	$b = \sum_{j=0}^D \beta_j z_j$	$g_{\beta_j} = g_b \frac{\partial b}{\partial \beta_j}, \frac{\partial b}{\partial \beta_j} = z_j$ $g_{z_j} = g_b \frac{\partial b}{\partial z_j}, \frac{\partial b}{\partial z_j} = \beta_j$
Sigmoid	$z_j = \frac{1}{1 + \exp(-a_j)}$	$g_{a_j} = g_{z_j} \frac{\partial z_j}{\partial a_j}, \frac{\partial z_j}{\partial a_j} = z_j(1 - z_j)$
Linear	$a_j = \sum_{i=0}^M \alpha_{ji} x_i$	$g_{\alpha_{ji}} = g_{a_j} \frac{\partial a_j}{\partial \alpha_{ji}}, \frac{\partial a_j}{\partial \alpha_{ji}} = x_i$ $g_{x_i} = \sum_{j=0}^D g_{a_j} \frac{\partial a_j}{\partial x_i}, \frac{\partial a_j}{\partial x_i} = \alpha_{ji}$

Backpropagation

Example:
Neural Network



Loss

Sigmoid

Linear

Sigmoid

Linear

This whole
“Backward” columns
is now computed for
us automatically by
AutoDiff

Backward

$$g_y = \frac{y^*}{y} + \frac{(1 - y^*)}{y - 1}$$

$$g_b = g_y \frac{\partial y}{\partial b}, \quad \frac{\partial y}{\partial b} = y(1 - y)$$

$$g_{\beta_j} = g_b \frac{\partial b}{\partial \beta_j}, \quad \frac{\partial b}{\partial \beta_j} = z_j$$

$$g_{z_j} = g_b \frac{\partial b}{\partial z_j}, \quad \frac{\partial b}{\partial z_j} = \beta_j$$

$$g_{a_j} = g_{z_j} \frac{\partial z_j}{\partial a_j}, \quad \frac{\partial z_j}{\partial a_j} = z_j(1 - z_j)$$

$$g_{\alpha_{ji}} = g_{a_j} \frac{\partial a_j}{\partial \alpha_{ji}}, \quad \frac{\partial a_j}{\partial \alpha_{ji}} = x_i$$

$$g_{x_i} = \sum_{j=0}^D g_{a_j} \frac{\partial a_j}{\partial x_i}, \quad \frac{\partial a_j}{\partial x_i} = \alpha_{ji}$$

$$a_j = \sum_{i=0}^M \alpha_{ji} x_i$$

$$z_j = \frac{1}{1 + \exp(-a_j)}$$

SGD with Backprop

Example: 1-Hidden Layer Neural Network

Algorithm 1 Stochastic Gradient Descent (SGD)

```
1: procedure SGD(Training data  $\mathcal{D}$ , test data  $\mathcal{D}_t$ )
2:   Initialize parameters  $\alpha, \beta$ 
3:   for  $e \in \{1, 2, \dots, E\}$  do
4:     for  $(\mathbf{x}, \mathbf{y}) \in \mathcal{D}$  do
5:       Compute neural network layers:
6:        $\mathbf{o} = \text{object}(\mathbf{x}, \mathbf{a}, \mathbf{b}, \mathbf{z}, \hat{\mathbf{y}}, J) = \text{NNFORWARD}(\mathbf{x}, \mathbf{y}, \alpha, \beta)$ 
7:       Compute gradients via backprop:
8:        $\left. \begin{array}{l} \mathbf{g}_\alpha = \nabla_\alpha J \\ \mathbf{g}_\beta = \nabla_\beta J \end{array} \right\} = \text{NNBACKWARD}(\mathbf{x}, \mathbf{y}, \alpha, \beta, \mathbf{o})$ 
9:       Update parameters:
10:       $\alpha \leftarrow \alpha - \gamma \mathbf{g}_\alpha$ 
11:       $\beta \leftarrow \beta - \gamma \mathbf{g}_\beta$ 
12:      Evaluate training mean cross-entropy  $J_{\mathcal{D}}(\alpha, \beta)$ 
13:      Evaluate test mean cross-entropy  $J_{\mathcal{D}_t}(\alpha, \beta)$ 
14:   return parameters  $\alpha, \beta$ 
```

SGD and Mini-batch SGD

Algorithm 1 SGD

```
1: Initialize  $\theta^{(0)}$ 
2:
3:
4:  $s = 0$ 
5: for  $t = 1, 2, \dots, T$  do
6:   for  $i \in \text{shuffle}(1, \dots, N)$  do
7:     Select the next training point  $(x_i, y_i)$ 
8:     Compute the gradient  $g^{(s)} = \nabla J_i(\theta^{(s-1)})$ 
9:     Update parameters  $\theta^{(s)} = \theta^{(s-1)} - \eta g^{(s)}$ 
10:    Increment time step  $s = s + 1$ 
11:    Evaluate average training loss  $J(\theta) = \frac{1}{n} \sum_{i=1}^n J_i(\theta)$ 
12: return  $\theta^{(s)}$ 
```

SGD and Mini-batch SGD

Algorithm 1 Mini-Batch SGD

- 1: Initialize $\theta^{(0)}$
 - 2: Divide examples $\{1, \dots, N\}$ randomly into batches $\{I_1, \dots, I_B\}$
 - 3: where $\bigcup_{b=1}^B I_b = \{1, \dots, N\}$ and $\bigcap_{b=1}^B I_b = \emptyset$
 - 4: $s = 0$
 - 5: **for** $t = 1, 2, \dots, T$ **do**
 - 6: **for** $b = 1, 2, \dots, B$ **do**
 - 7: Select the next batch I_b , where $m = |I_b|$
 - 8: Compute the gradient $g^{(s)} = \frac{1}{m} \sum_{i \in I_b} \nabla J_i(\theta^{(s)})$
 - 9: Update parameters $\theta^{(s)} = \theta^{(s-1)} - \eta g^{(s)}$
 - 10: Increment time step $s = s + 1$
 - 11: Evaluate average training loss $J(\theta) = \frac{1}{n} \sum_{i=1}^n J_i(\theta)$
 - 12: **return** $\theta^{(s)}$
-

LEARNING AN RNN FOR SEQUENCE TAGGING

Dataset for Supervised Part-of-Speech (POS) Tagging

Data: $\mathcal{D} = \{\mathbf{x}^{(n)}, \mathbf{y}^{(n)}\}_{n=1}^N$

Sample 1:	n time	v flies	p like	d an	n arrow	} $y^{(1)}$ } $x^{(1)}$
Sample 2:	n time	n flies	v like	d an	n arrow	} $y^{(2)}$ } $x^{(2)}$
Sample 3:	n flies	v fly	p with	n their	n wings	} $y^{(3)}$ } $x^{(3)}$
Sample 4:	p with	n time	n you	v will	v see	} $y^{(4)}$ } $x^{(4)}$

Dataset for Supervised Handwriting Recognition

Data: $\mathcal{D} = \{x^{(n)}, y^{(n)}\}_{n=1}^N$



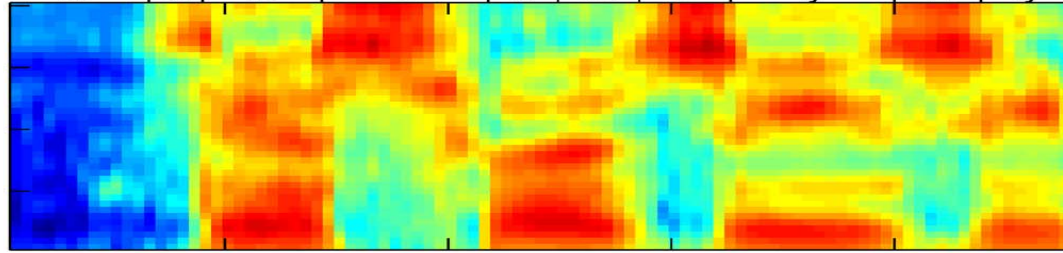
Dataset for Supervised Phoneme (Speech) Recognition

Data: $\mathcal{D} = \{\mathbf{x}^{(n)}, \mathbf{y}^{(n)}\}_{n=1}^N$

Sample 1:



} $\mathbf{y}^{(1)}$

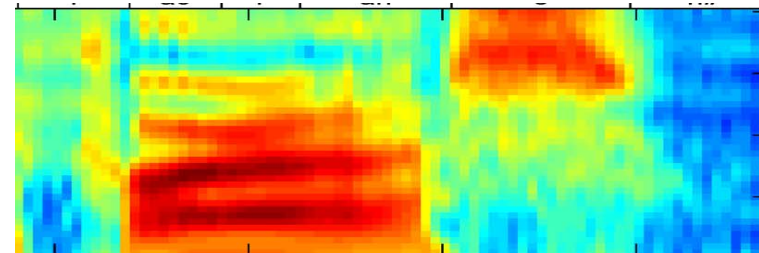


} $\mathbf{x}^{(1)}$

Sample 2:



} $\mathbf{y}^{(2)}$



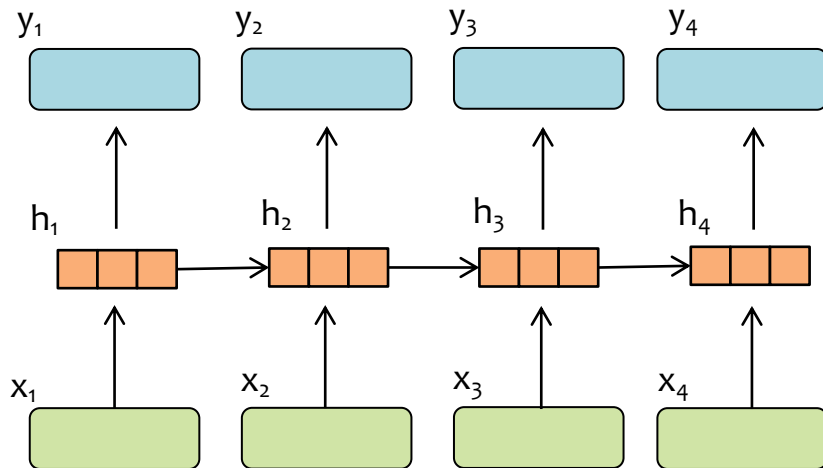
} $\mathbf{x}^{(2)}$

RNN

Question: where do these parameter tensors come from?

Algorithm 1 Elman RNN

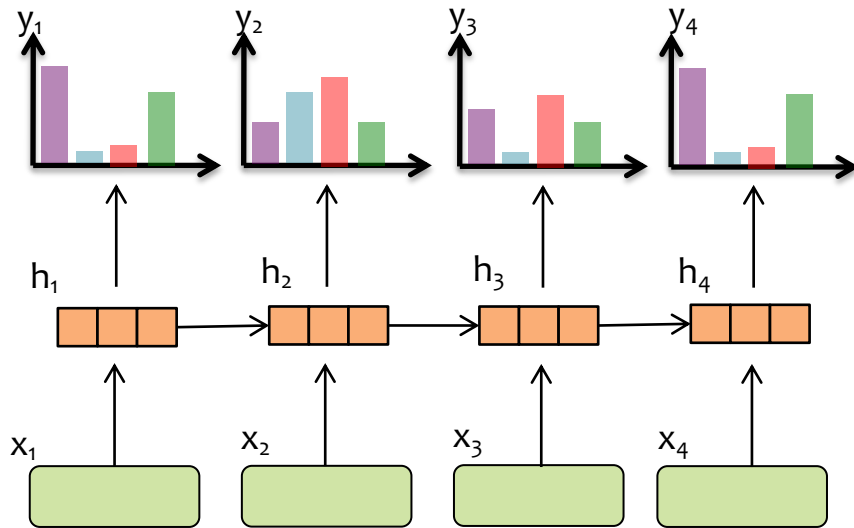
- 1: **procedure** FORWARD($x_{1:T}, W_{ah}, W_{ax}, b_a, W_{yh}, b_y$)
- 2: Initialize the hidden state h_0 to zeros
- 3: **for** t in 1 to T **do**
- 4: Receive input data at time step t : x_t
- 5: Compute the hidden state update:
- 6: $a_t = W_{ah} \cdot h_{t-1} + W_{ax} \cdot x_t + b_a$
- 7: $h_t = \sigma(a_t)$
- 8: Compute the output at time step t :
- 9: $y_t = W_{yh} \cdot h_t + b_y$



RNN

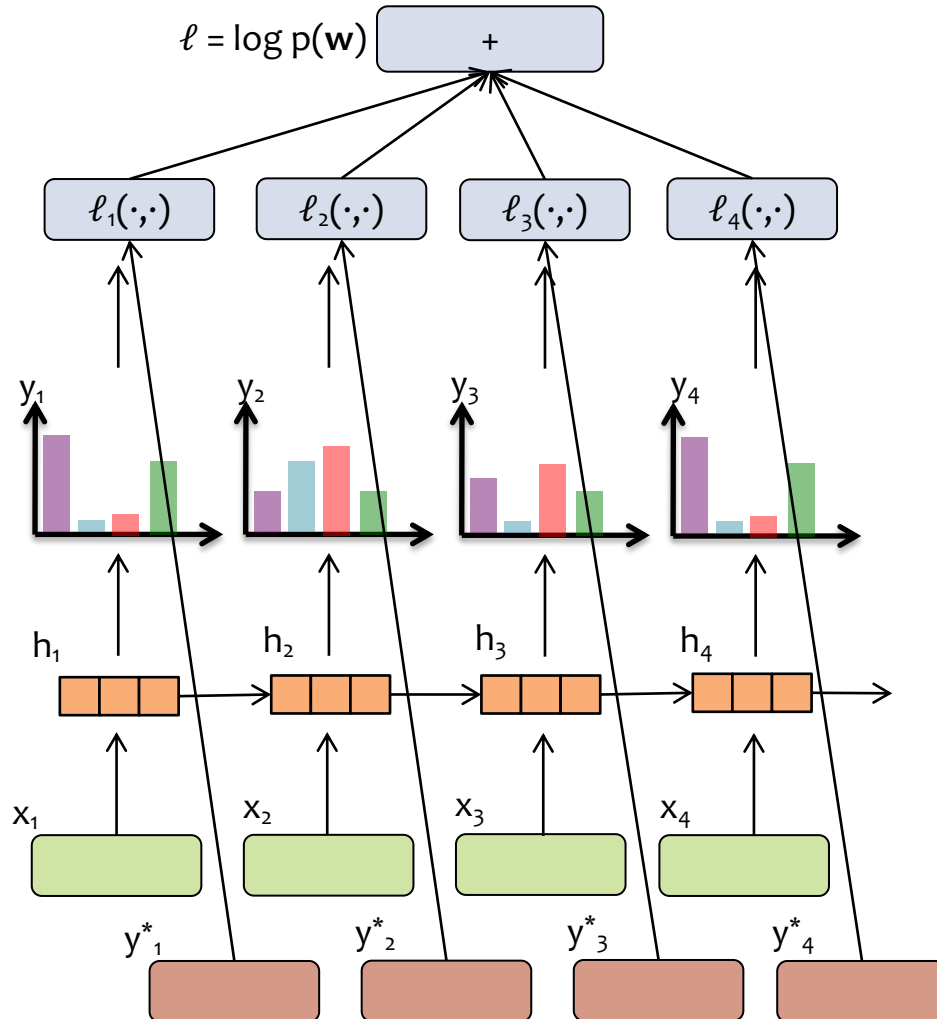
Algorithm 1 Elman RNN

- 1: **procedure** FORWARD($x_{1:T}, W_{ah}, W_{ax}, b_a, W_{yh}, b_y$)
 - 2: Initialize the hidden state h_0 to zeros
 - 3: **for** t in 1 to T **do**
 - 4: Receive input data at time step t : x_t
 - 5: Compute the hidden state update:
 - 6: $a_t = W_{ah} \cdot h_{t-1} + W_{ax} \cdot x_t + b_a$
 - 7: $h_t = \sigma(a_t)$
 - 8: Compute the output at time step t :
 - 9: $y_t = \text{softmax}(W_{yh} \cdot h_t + b_y)$
-



RNN + Loss

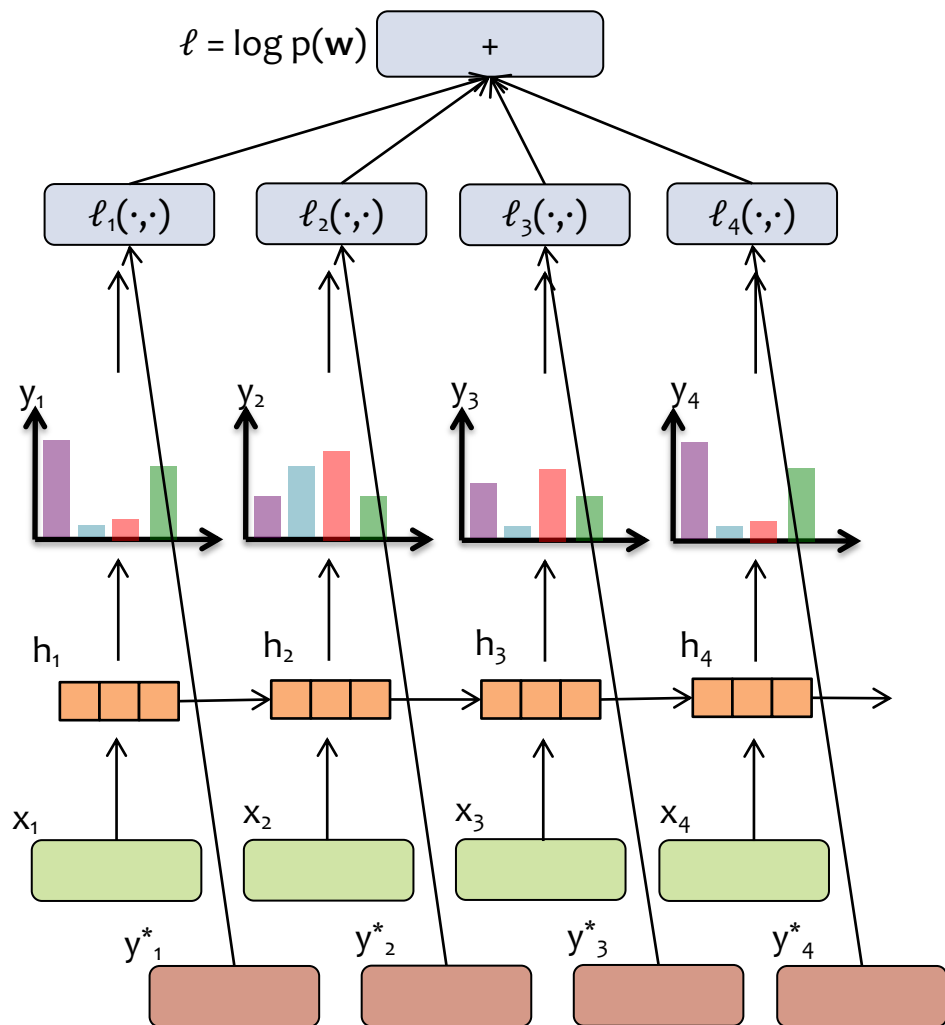
Algorithm 1 Elman RNN + Loss



- 1: **procedure** FORWARD($x_{1:T}, y_{1:T}^*, W_{ah}, W_{ax}, b_a, W_{yh}, b_y$)
 - 2: Initialize the hidden state h_0 to zeros
 - 3: **for** t in 1 to T **do**
 - 4: Receive input data at time step t : x_t
 - 5: Compute the hidden state update:
 - 6: $a_t = W_{ah} \cdot h_{t-1} + W_{ax} \cdot x_t + b_a$
 - 7: $h_t = \sigma(a_t)$
 - 8: Compute the output at time step t :
 - 9: $y_t = \text{softmax}(W_{yh} \cdot h_t + b_y)$
 - 10: Compute the cross-entropy loss at time step t :
 - 11: $\ell_t = - \sum_{k=1}^K (y_t^*)_k \log((y_t)_k)$
 - 12: Compute the total loss:
 - 13: $\ell = \sum_{t=1}^T \ell_t$
-

RNN + Loss

Algorithm 1 Elman RNN + Loss



- 1: **procedure** FORWARD($x_{1:T}, y_{1:T}^*, W_{ah}, W_{ax}, b_a, W_{yh}, b_y$)
- 2: Initialize the hidden state h_0 to zeros
- 3: **for** t in 1 to T **do**
- 4: Receive input data at time step t : x_t
- 5: Compute the hidden state update:
- 6: $a_t = W_{ah} \cdot h_{t-1} + W_{ax} \cdot x_t + b_a$
- 7: $h_t = \sigma(a_t)$
- 8: Compute the output at time step t :
- 9: $y_t = \text{softmax}(W_{yh} \cdot h_t + b_y)$
- 10: Compute the cross-entropy loss at time step t :
- 11: $\ell_t = - \sum_{k=1}^K (y_t^*)_k \log((y_t)_k)$
- 12: Compute the total loss:
- 13: $\ell = \sum_{t=1}^T \ell_t$

Which of these steps could we skip in a real implementation for efficiency?

(e.g. RNN-LM or Transformer LM)

LEARNING A DEEP NEURAL LM

Dataset for Language Modeling

Data: $\mathcal{D} = \{\mathbf{x}^{(n)}\}_{n=1}^N$

Sample 1:	time	flies	like	an	arrow	} $\mathbf{x}^{(1)}$
Sample 2:	the	cows	eat	grass		} $\mathbf{x}^{(2)}$
Sample 3:	birds	fly	fast			} $\mathbf{x}^{(3)}$
Sample 4:	the	cat	saw	donuts		} $\mathbf{x}^{(4)}$

...

Learning a Language Model

Question: How do we **learn** the probabilities for the n-Gram Model?

Answer: From data! Just **count** n-gram frequencies

... the **cows** eat **grass**...
... our **cows** eat **hay** daily...
... factory-farm **cows** eat **corn**...
... on an organic farm, **cows** eat **hay** and...
... do your **cows** eat **grass** or corn?...
... what do **cows** eat if they have...
... **cows** eat **corn** when there is no...
... which **cows** eat which foods depends...
... if **cows** eat **grass**...
... when **cows** eat **corn** their stomachs...
... should we let **cows** eat **corn**?...

$$p(w_t \mid w_{t-2} = \text{cows}, w_{t-1} = \text{eat})$$


w_t	$p(\cdot \mid \cdot, \cdot)$
corn	4/11
grass	3/11
hay	2/11
if	1/11
which	1/11

MLE for n-gram LM

- This counting method gives us the **maximum likelihood estimate** of the n-gram LM parameters
- We can derive it in the usual way:
 - **Write the likelihood** of the sentences under the n-gram LM
 - **Set the gradient to zero** and impose the constraint that the probabilities sum-to-one
 - **Solve** for the MLE

Learning a Language Model

MLE for Deep Neural LM

- We can also use maximum likelihood estimation to learn the parameters of an RNN-LM or Transformer-LM too!
- But **not in closed form** – instead we follow a different recipe:
 - Write the **likelihood** of the sentences under the Deep Neural LM model
 - Compute the **gradient** of the (batch) likelihood w.r.t. the parameters **by AutoDiff**
 - Follow the negative gradient using **Mini-batch SGD** (or your favorite optimizer)

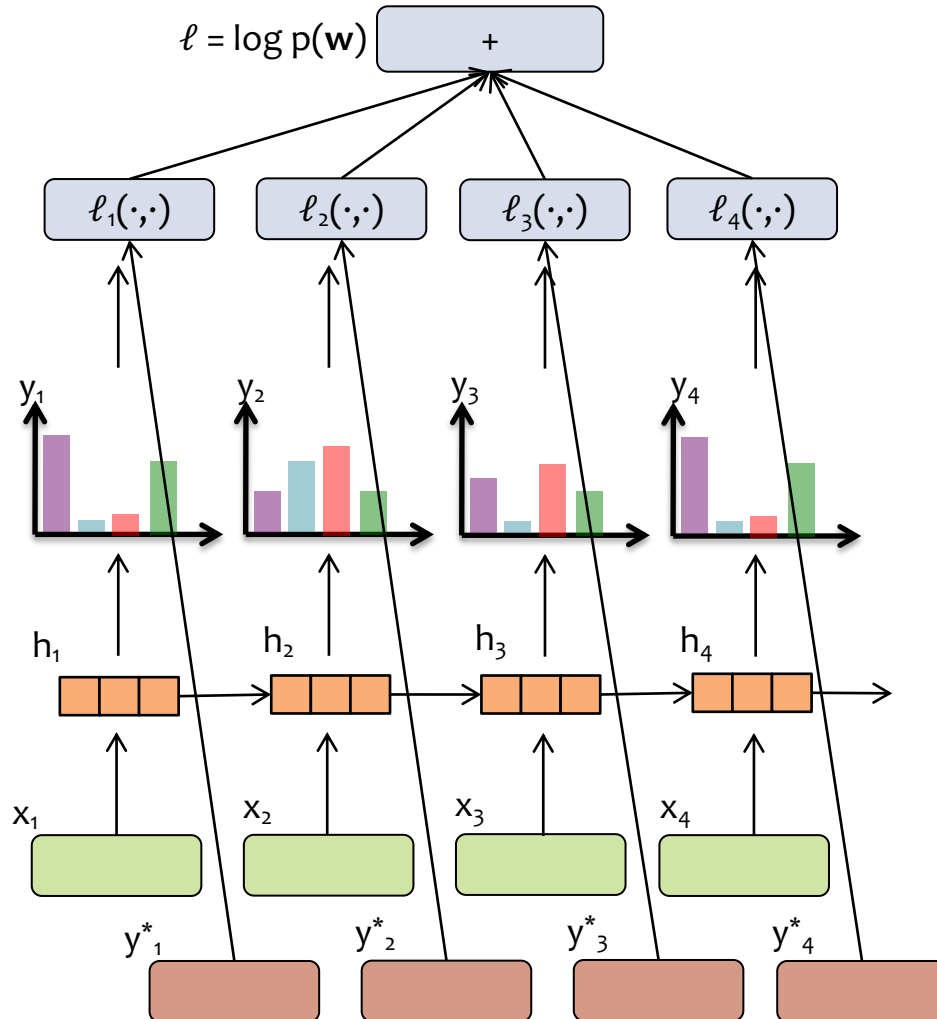
MLE for n-gram LM

- This counting method gives us the **maximum likelihood estimate** of the n-gram LM parameters
- We can derive it in the usual way:
 - **Write the likelihood** of the sentences under the n-gram LM
 - **Set the gradient to zero** and impose the constraint that the probabilities sum-to-one
 - **Solve** for the MLE

RNN + LOSS

How can we use this to compute the loss for an RNN-LM?

Algorithm 1 Elman RNN + Loss

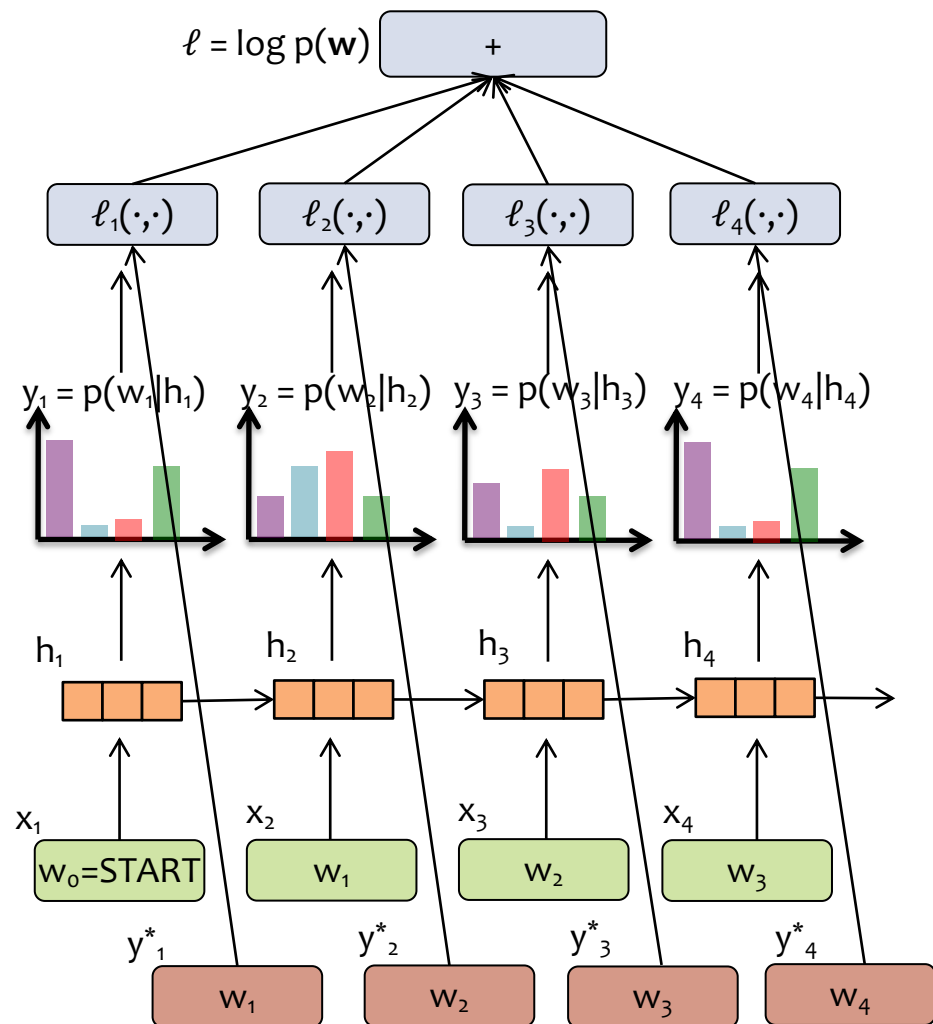


- 1: **procedure** FORWARD($x_{1:T}, y_{1:T}^*, W_{ah}, W_{ax}, b_a, W_{yh}, b_y$)
- 2: Initialize the hidden state h_0 to zeros
- 3: **for** t in 1 to T **do**
- 4: Receive input data at time step t : x_t
- 5: Compute the hidden state update:
- 6: $a_t = W_{ah} \cdot h_{t-1} + W_{ax} \cdot x_t + b_a$
- 7: $h_t = \sigma(a_t)$
- 8: Compute the output at time step t :
- 9: $y_t = \text{softmax}(W_{yh} \cdot h_t + b_y)$
- 10: Compute the cross-entropy loss at time step t :
- 11: $\ell_t = - \sum_{k=1}^K (y_t^*)_k \log((y_t)_k)$
- 12: Compute the total loss:
- 13: $\ell = \sum_{t=1}^T \ell_t$

RNN-LM + LOSS

How can we use this to compute the loss for an RNN-LM?

$$\begin{aligned}\log p(\mathbf{w}) &= \log p(w_1, w_2, w_3, \dots, w_T) \\ &= \log p(w_1 | h_1) + \dots + \log p(w_T | h_T)\end{aligned}$$



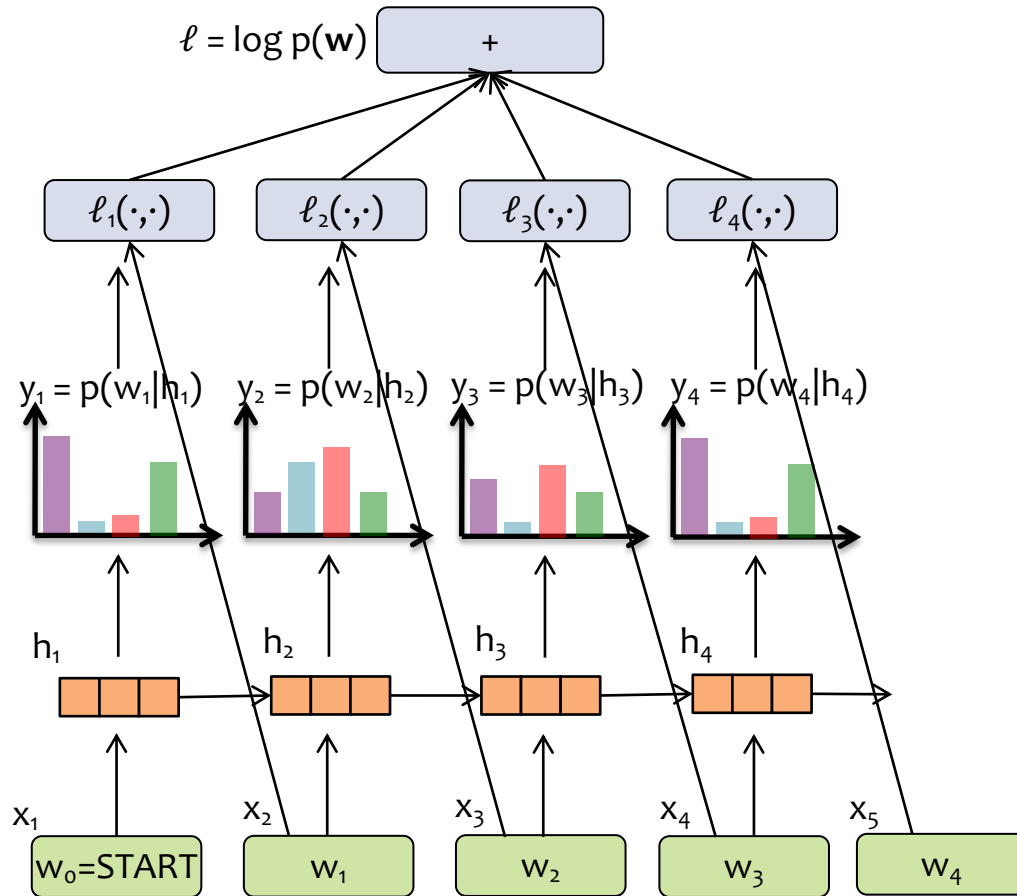
Algorithm 1 Elman RNN + Loss

- 1: **procedure** FORWARD($x_{1:T}, y_{1:T}^*, W_{ah}, W_{ax}, b_a, W_{yh}, b_y$)
- 2: Initialize the hidden state h_0 to zeros
- 3: **for** t in 1 to T **do**
- 4: Receive input data at time step t : x_t
- 5: Compute the hidden state update:
- 6: $a_t = W_{ah} \cdot h_{t-1} + W_{ax} \cdot x_t + b_a$
- 7: $h_t = \sigma(a_t)$
- 8: Compute the output at time step t :
- 9: $y_t = \text{softmax}(W_{yh} \cdot h_t + b_y)$
- 10: Compute the cross-entropy loss at time step t :
- 11: $\ell_t = - \sum_{k=1}^K (y_t^*)_k \log((y_t)_k)$
- 12: Compute the total loss:
- 13: $\ell = \sum_{t=1}^T \ell_t$

RNN-LM + LOSS

How can we use this to compute the loss for an RNN-LM?

$$\begin{aligned}\log p(\mathbf{w}) &= \log p(w_1, w_2, w_3, \dots, w_T) \\ &= \log p(w_1 | h_1) + \dots + \log p(w_T | h_T)\end{aligned}$$

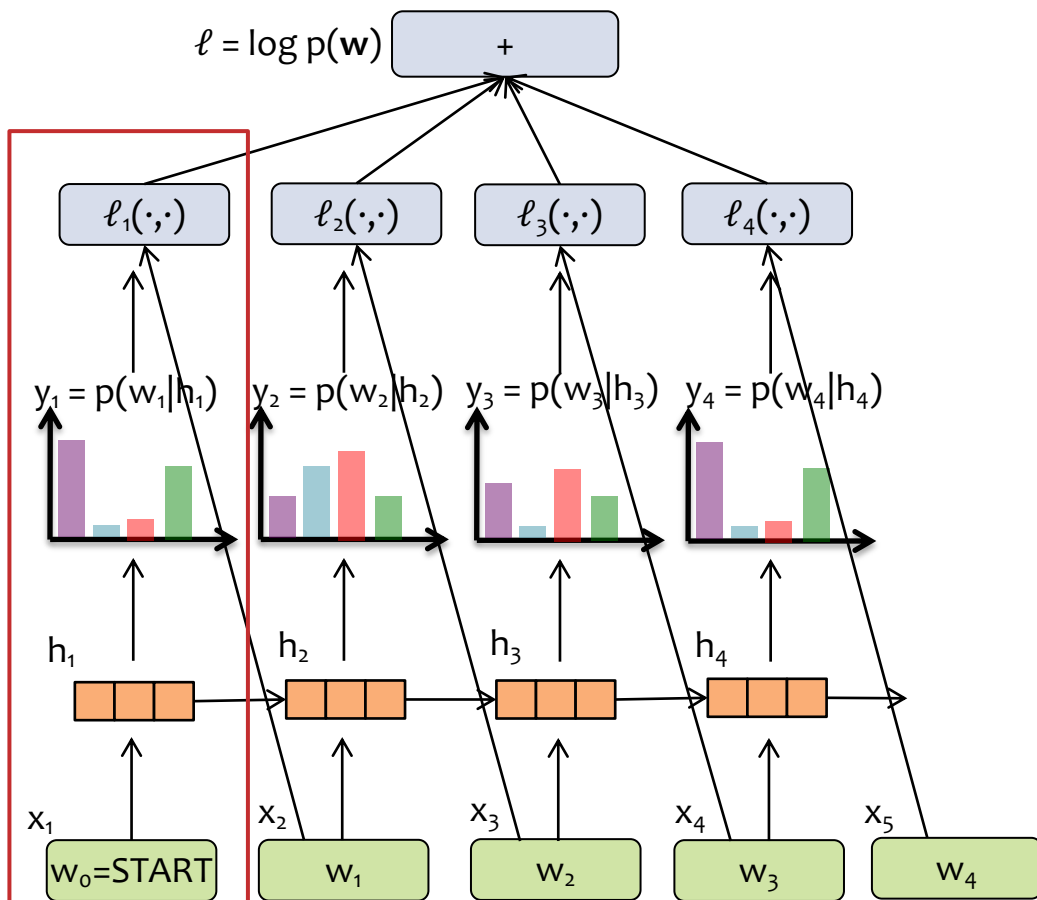


Algorithm 1 Elman RNN + Loss

- 1: **procedure** FORWARD($x_{1:T}, y_{1:T}^*, W_{ah}, W_{ax}, b_a, W_{yh}, b_y$)
- 2: Initialize the hidden state h_0 to zeros
- 3: **for** t in 1 to T **do**
- 4: Receive input data at time step t : x_t
- 5: Compute the hidden state update:
- 6: $a_t = W_{ah} \cdot h_{t-1} + W_{ax} \cdot x_t + b_a$
- 7: $h_t = \sigma(a_t)$
- 8: Compute the output at time step t :
- 9: $y_t = \text{softmax}(W_{yh} \cdot h_t + b_y)$
- 10: Compute the cross-entropy loss at time step t :
- 11: $\ell_t = - \sum_{k=1}^K (y_t^*)_k \log((y_t)_k)$
- 12: Compute the total loss:
- 13: $\ell = \sum_{t=1}^T \ell_t$

RNN-LM + Loss

$$\begin{aligned}\log p(\mathbf{w}) &= \log p(w_1, w_2, w_3, \dots, w_T) \\ &= \log p(w_1 | h_1) + \dots + \log p(w_T | h_T)\end{aligned}$$



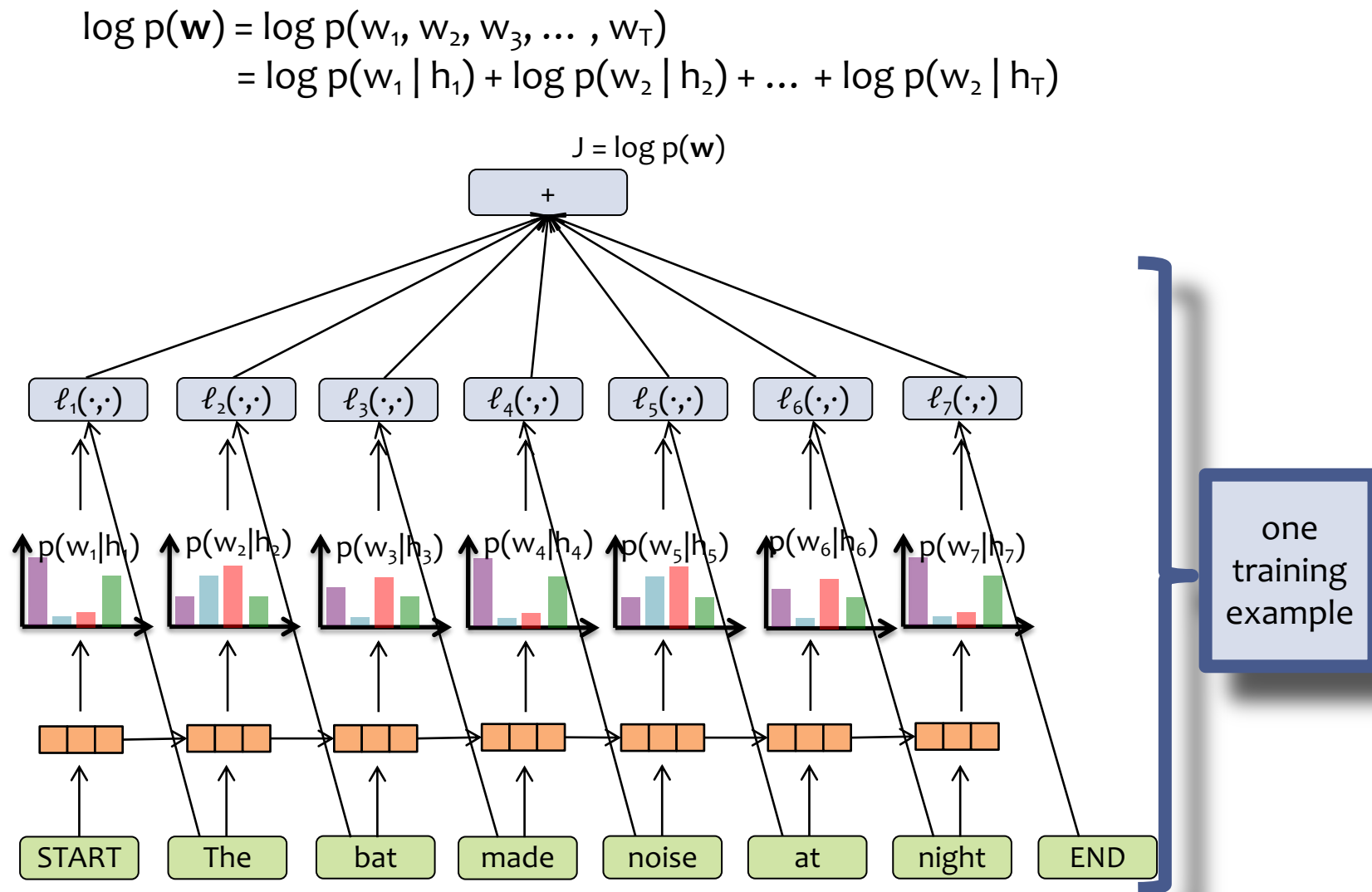
Question: If we removed this entire column, what probability distribution would we be defining?

Algorithm 1 Elman RNN + Loss

- 1: **procedure** FORWARD($x_{1:T}, y_{1:T}^*, W_{ah}, W_{ax}, b_a, W_{yh}, b_y$)
- 2: Initialize the hidden state h_0 to zeros
- 3: **for** t in 1 to T **do**
- 4: Receive input data at time step t : x_t
- 5: Compute the hidden state update:
- 6: $a_t = W_{ah} \cdot h_{t-1} + W_{ax} \cdot x_t + b_a$
- 7: $h_t = \sigma(a_t)$
- 8: Compute the output at time step t :
- 9: $y_t = \text{softmax}(W_{yh} \cdot h_t + b_y)$
- 10: Compute the cross-entropy loss at time step t :
- 11: $\ell_t = -\sum_{k=1}^K (y_t^*)_k \log((y_t)_k)$
- 12: Compute the total loss:
- 13: $\ell = \sum_{t=1}^T \ell_t$

Learning an RNN-LM

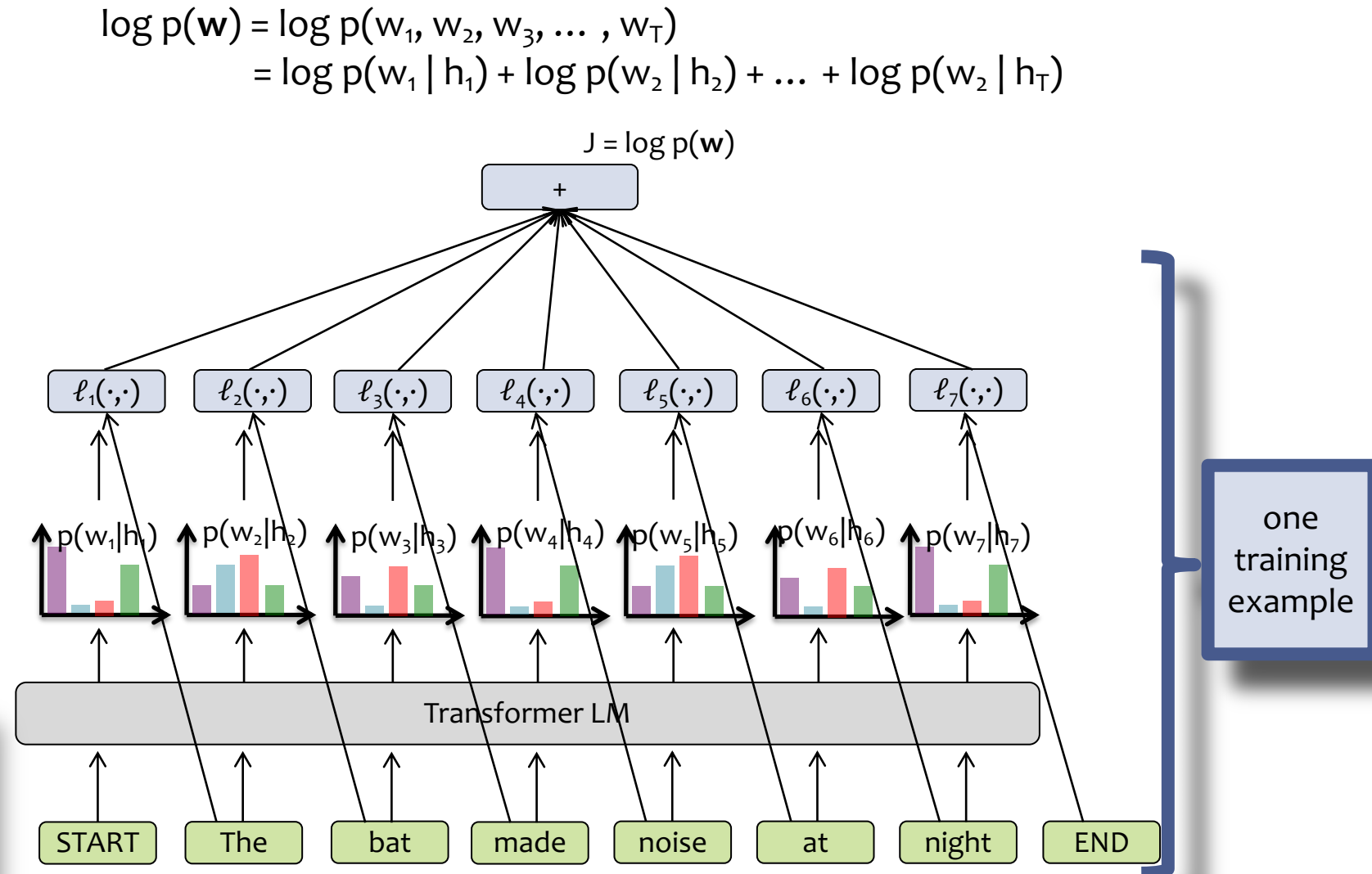
- Each training example is a sequence (e.g. sentence), so we have training data $D = \{\mathbf{w}^{(1)}, \mathbf{w}^{(2)}, \dots, \mathbf{w}^{(N)}\}$
- The objective function for a Deep LM (e.g. RNN-LM or Transformer-LM) is typically the log-likelihood of the training examples:
$$J(\boldsymbol{\theta}) = \sum_i \log p_{\boldsymbol{\theta}}(\mathbf{w}^{(i)})$$
- We train by mini-batch SGD (or your favorite flavor of mini-batch SGD)



Learning a Transformer LM

- Each training example is a sequence (e.g. sentence), so we have training data $D = \{\mathbf{w}^{(1)}, \mathbf{w}^{(2)}, \dots, \mathbf{w}^{(N)}\}$
- The objective function for a Deep LM (e.g. RNN-LM or Transformer-LM) is typically the log-likelihood of the training examples:
$$J(\theta) = \sum_i \log p_{\theta}(\mathbf{w}^{(i)})$$
- We train by mini-batch SGD (or your favorite flavor of mini-batch SGD)

Training a Transformer-LM is the same, except we swap in a different deep language model.



Language Modeling

An aside:

- State-of-the-art language models currently tend to rely on **transformer networks** (e.g. GPT-2)
- RNN-LMs comprised most of the early neural LMs that **led to** current SOTA architectures

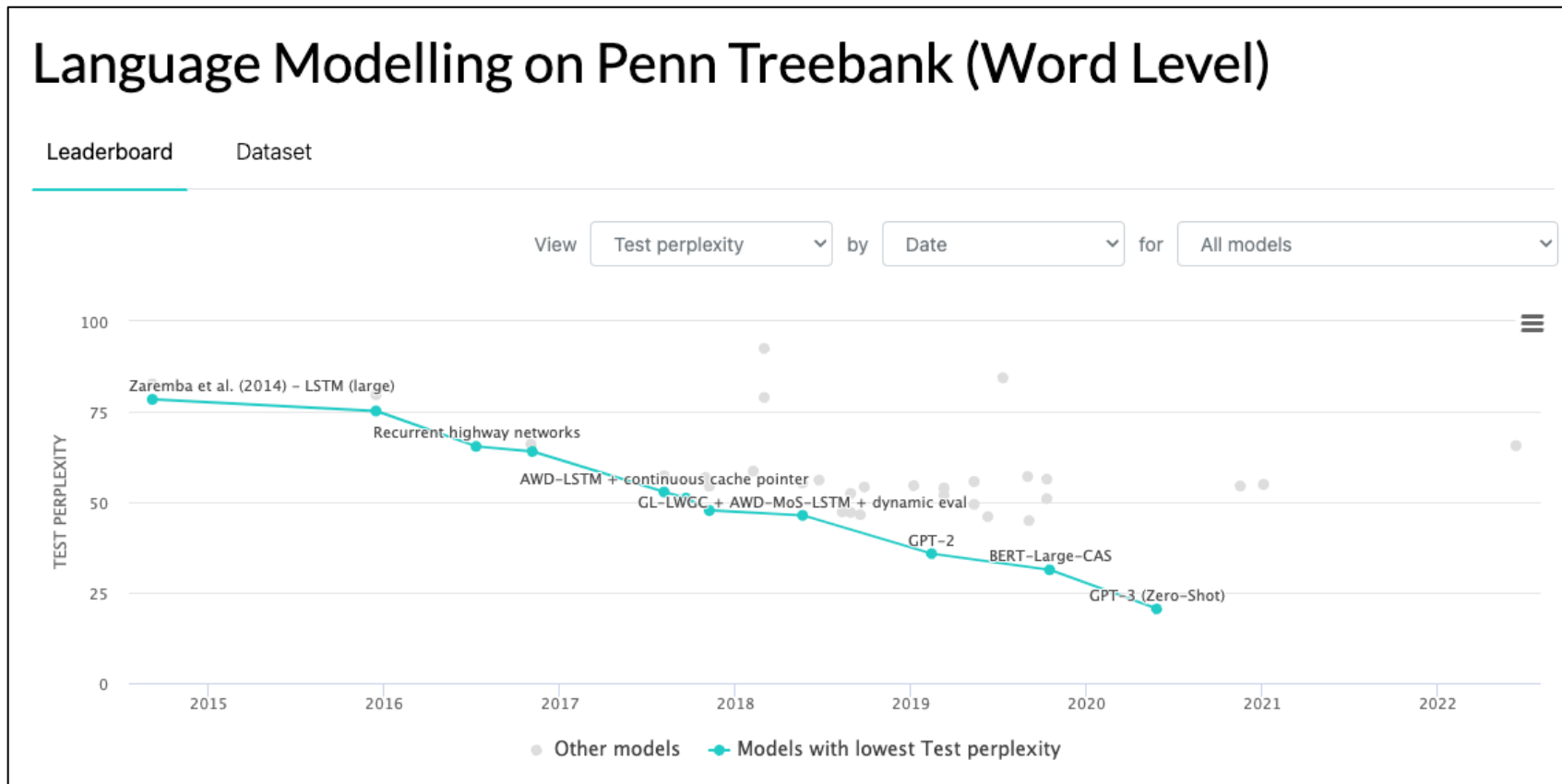


Figure from <https://paperswithcode.com/sota/language-modelling-on-penn-treebank-word>

BATCHING, PADDING, & TRUNCATION

Why does efficiency matter?

Case Study: GPT-3

- # of training tokens = 500 billion
- # of parameters = 175 billion
- # of cycles = 50 petaflop/s-days (each of which are 8.64×10^{19} flops)

Dataset	Quantity (tokens)	Weight in training mix	Epochs elapsed when training for 300B tokens
Common Crawl (filtered)	410 billion	60%	0.44
WebText2	19 billion	22%	2.9
Books1	12 billion	8%	1.9
Books2	55 billion	8%	0.43
Wikipedia	3 billion	3%	3.4

Table 2.2: Datasets used to train GPT-3. “Weight in training mix” refers to the fraction of examples during training that are drawn from a given dataset, which we intentionally do not make proportional to the size of the dataset. As a result, when we train for 300 billion tokens, some datasets are seen up to 3.4 times during training while other datasets are seen less than once.

Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}	Batch Size	Learning Rate
GPT-3 Small	125M	12	768	12	64	0.5M	6.0×10^{-4}
GPT-3 Medium	350M	24	1024	16	64	0.5M	3.0×10^{-4}
GPT-3 Large	760M	24	1536	16	96	0.5M	2.5×10^{-4}
GPT-3 XL	1.3B	24	2048	24	128	1M	2.0×10^{-4}
GPT-3 2.7B	2.7B	32	2560	32	80	1M	1.6×10^{-4}
GPT-3 6.7B	6.7B	32	4096	32	128	2M	1.2×10^{-4}
GPT-3 13B	13.0B	40	5140	40	128	2M	1.0×10^{-4}
GPT-3 175B or “GPT-3”	175.0B	96	12288	96	128	3.2M	0.6×10^{-4}

Table 2.1: Sizes, architectures, and learning hyper-parameters (batch size in tokens and learning rate) of the models which we trained. All models were trained for a total of 300 billion tokens.

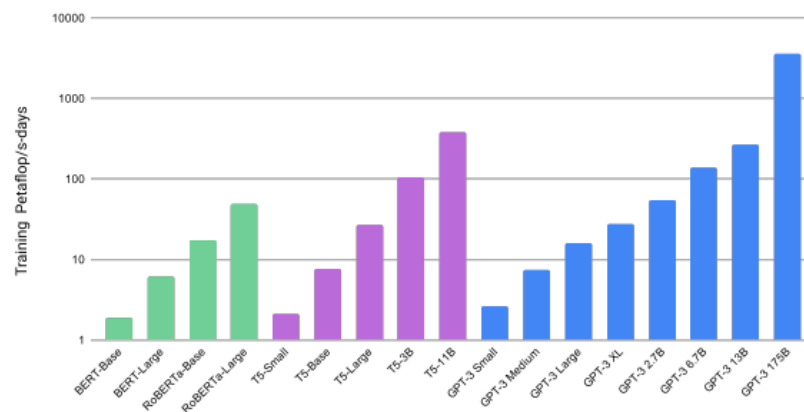


Figure 2.2: Total compute used during training. Based on the analysis in Scaling Laws For Neural Language Models [KMH⁺20] we train much larger models on many fewer tokens than is typical. As a consequence, although GPT-3 3B is almost 10x larger than RoBERTa-Large (355M params), both models took roughly 50 petaflop/s-days of compute during pre-training. Methodology for these calculations can be found in Appendix D.

Efficient Parallelism for Transformers

Transformers can be trained very efficiently!
(This is arguably one of the key reasons they have been so successful.)

- **Batching:** Rather than processing one sentence at a time, Transformers take in a batch of B sentences at a time. The computation is identical for each batch and is trivially parallelized.
- **Scaled Dot-product Attention:** can be easily parallelized because the attention scores of one timestep do not depend on other timesteps.
- **Multi-headed Attention:** computes each head independently, which permits yet more parallelism.
- **Matrix multiplication:** The core computation in attention is matrix multiplication, and specialized hardware (GPUs and TPUs) makes this very fast.
- **Model parallelism:** For huge models, we can divide the model over multiple GPUs/machines.
- **Key-value caching:** The keys and values are re-used over many timesteps, but we do not need to cache the queries, similarity scores, and attention weights.

Batching: Padding and Truncation

- Suppose we have 8 training sentences
- We set our block size (maximum sequence length) to 10
- Before collecting them into a batch, we:
 1. truncate those sentences that are too long
 2. pad the sentences that are too short
 3. convert each token to an integer via a lookup table (vocabulary)
 4. convert each token to an embedding vector of fixed length

i	w_1	w_2	w_3	w_4	w_5	w_6	w_7	w_8	w_9	w_{10}	w_{11}	w_{12}
1	In	the	hole	in	the	ground	there	lived	a	hobbit		
2	It	is	our	choices	that	show	what	we	truly	are		
3	It	was	the	best	of	times	it	was	the	worst	of	times
4	Even	miracles	take	a	little	time						
5	The	more	that	you	read	the	more	things	you	will	know	
6	We'll	always	have	each	other	no	matter	what	happens			
7	The	sun	did	not	shine	it	was	too	wet	to	play	
8	The	important	thing	is	to	never	stop	questioning				

Batching: Padding and Truncation

- Suppose we have 8 training sentences
- We set our block size (maximum sequence length) to 10
- Before collecting them into a batch, we:
 1. truncate those sentences that are too long
 2. pad the sentences that are too short
 3. convert each token to an integer via a lookup table (vocabulary)
 4. convert each token to an embedding vector of fixed length

i	w ₁	w ₂	w ₃	w ₄	w ₅	w ₆	w ₇	w ₈	w ₉	w ₁₀	w ₁₁	w ₁₂
1	In	the	hole	in	the	ground	there	lived	a	hobbit		
2	It	is	our	choices	that	show	what	we	truly	are		
3	It	was	the	best	of	times	it	was	the	worst	of	times
4	Even	miracles	take	a	little	time	<PAD>	<PAD>	<PAD>	<PAD>		
5	The	more	that	you	read	the	more	things	you	will	know	
6	We'll	always	have	each	other	no	matter	what	happens	<PAD>		
7	The	sun	did	not	shine	it	was	too	wet	to	play	
8	The	important	thing	is	to	never	stop	questioning	<PAD>	<PAD>		

Batching: Padding and Truncation

- Suppose we have 8 training sentences
- We set our block size (maximum sequence length) to 10
- Before collecting them into a batch, we:
 1. truncate those sentences that are too long
 2. pad the sentences that are too short
 3. convert each token to an integer via a lookup table (vocabulary)
 4. convert each token to an embedding vector of fixed length

i	w ₁	w ₂	w ₃	w ₄	w ₅	w ₆	w ₇	w ₈	w ₉	w ₁₀
1	2	41	17	19	41	13	42	23	6	16
2	3	20	32	10	40	36	53	51	49	8
3	3	50	41	9	30	46	21	50	41	55
4	1	25	39	6	22	45	0	0	0	0
5	4	26	40	56	34	41	26	44	56	54
6	5	7	15	12	31	28	24	53	14	0
7	4	38	11	29	35	21	50	48	52	47
8	4	18	43	20	47	27	37	33	0	0

Vocabulary:

```
{  
    '<PAD>': 0,  
    'Even': 1,  
    'In': 2,  
    'It': 3,  
    'The': 4,  
    'We'll': 5,  
    'a': 6,  
    'always': 7,  
    'are': 8,  
    'best': 9,  
    ...  
    'what': 53,  
    'will': 54,  
    'worst': 55,  
    'you': 56  
}
```

Batching: Padding and Truncation

- Suppose we have 8 training sentences
- We set our block size (maximum sequence length) to 10
- Before collecting them into a batch, we:
 1. truncate those sentences that are too long
 2. pad the sentences that are too short
 3. convert each token to an integer via a lookup table (vocabulary)
 4. convert each token to an embedding vector of fixed length

i	w ₁	w ₂	w ₃	w ₄	w ₅	w ₆	w ₇	w ₈	w ₉	w ₁₀
1										
2										
3										
4										
5										
6										
7										
8										

Embeddings:

```
{  
  0 :   
  1 :   
  2 :   
  3 :   
  4 :   
  5 :   
  6 :   
  7 :   
  ...  
  55 :   
  56 :   
}
```

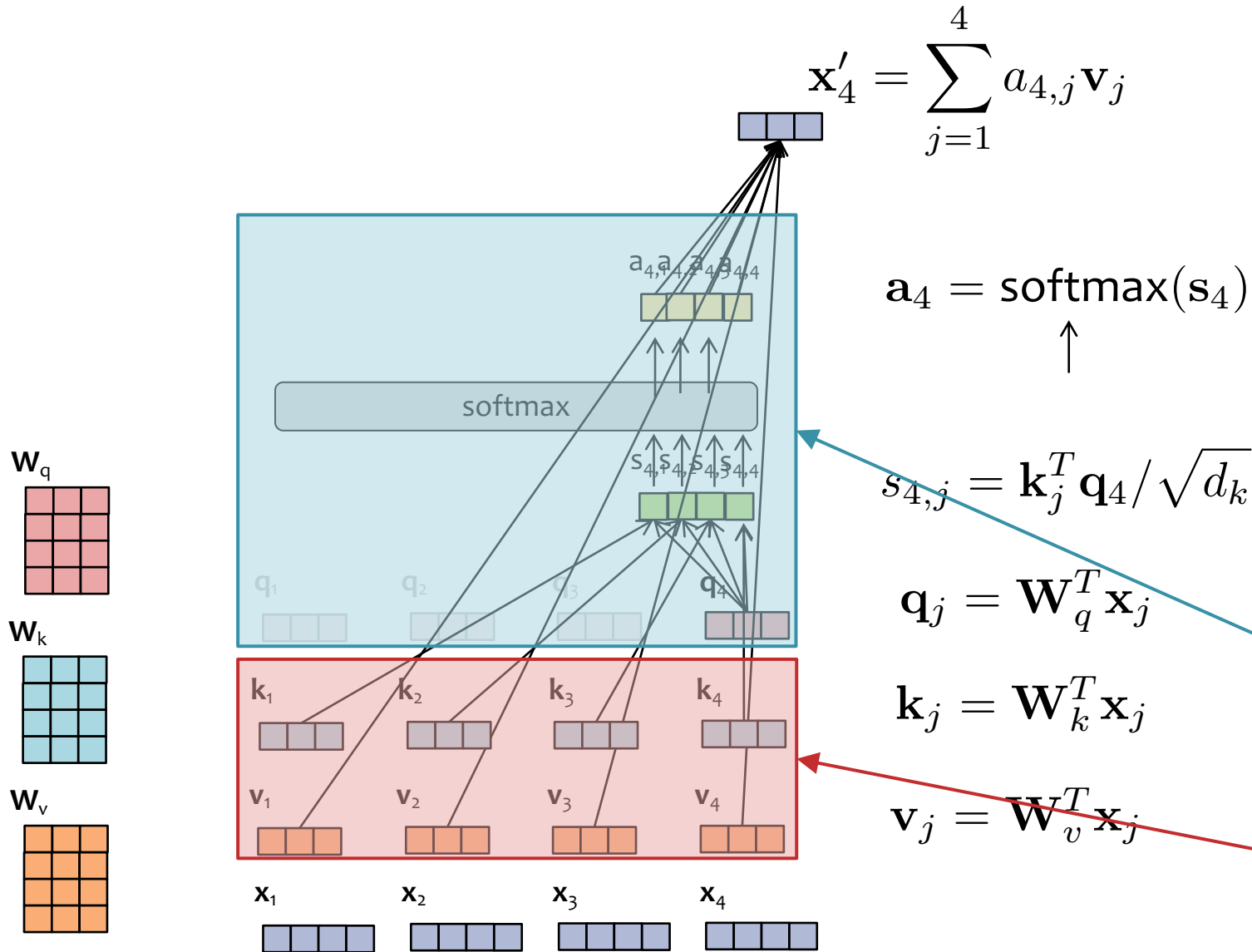
Efficient Parallelism for Transformers

Transformers can be trained very efficiently!
(This is arguably one of the key reasons they have been so successful.)

- **Batching:** Rather than processing one sentence at a time, Transformers take in a batch of B sentences at a time. The computation is identical for each batch and is trivially parallelized.
- **Scaled Dot-product Attention:** can be easily parallelized because the attention scores of one timestep do not depend on other timesteps.
- **Multi-headed Attention:** computes each head independently, which permits yet more parallelism.
- **Matrix multiplication:** The core computation in attention is matrix multiplication, and specialized hardware (GPUs and TPUs) makes this very fast.
- **Model parallelism:** For huge models, we can divide the model over multiple GPUs/machines.
- **Key-value caching:** The keys and values are re-used over many timesteps, but we do not need to cache the queries, similarity scores, and attention weights.

KV CACHE

Key-Value Cache



During generation, at each timestep:

- we reuse all previous keys and values (i.e. we need to cache them)
- but we can get rid of the queries, similarity scores, and attention weights

Discarded after this timestep

The KV Cache: each time we compute the new $\mathbf{k}_j$ and $\mathbf{v}_j$ for this timestep and cache them for subsequent timesteps

TOKENIZATION

Tokenization

Word-based Tokenizer:

Input: “Matt is giving a lecture on transformers”

Output: [“matt”, “is”, “giving”, “a”, “lecture”, “on”, “transformers”]

Pros/Cons:

- Can have difficulty trading off between vocabulary size and computational tractability
- Similar words e.g., “transformers” and “transformer” can get mapped to completely disparate representations
- Typos will typically be out-of-vocabulary (OOV)

Tokenization

Word-based Tokenizer:

Input: “Matt is givin’ a lectrue on transformers”

Output: [“matt”, “is”, <OOV>, “a”, <OOV>, “on”, “transformers”]

Pros/Cons:

- Can have difficulty trading off between vocabulary size and computational tractability
- Similar words e.g., “transformers” and “transformer” can get mapped to completely disparate representations
- Typos will typically be out-of-vocabulary (OOV)

Tokenization

Character-based Tokenizer:

Input: “Matt is givin’ a lectrue on transformers”

Output: [“m”, “a”, “t”, “t”, “ ”, “i”, “s”, “ ”, “g”, “i”, “v”, “i”, “n”, “ ’ ”, ...]

Pros/Cons:

- Much smaller vocabularies but a lot of semantic meaning is lost...
- Sequences will be much longer than word-based tokenization, potentially causing computational issues
- Can do well on logographic languages e.g., Kanji 漢字

Tokenization

Subword-based Tokenizer:

Input: “Matt is givin’ a lectrue on transformers”

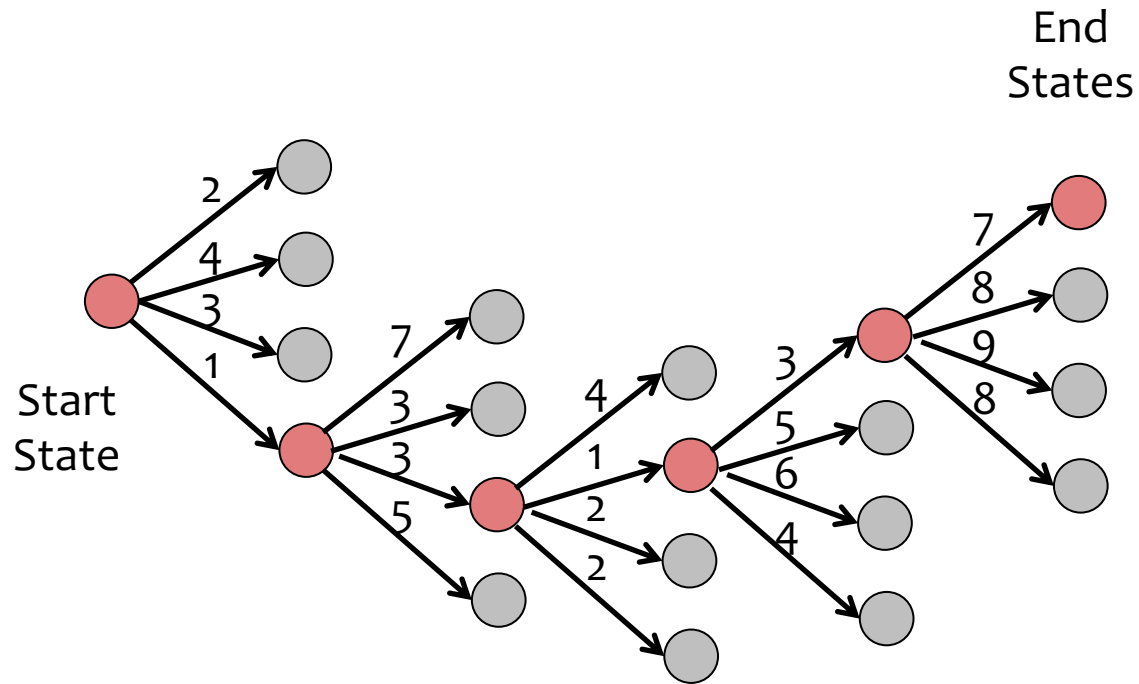
Output: [“matt”, “is”, “giv”, “##in”, “ ’ ”, “a”, “lec” “##true”, “on”, “transform”, “##ers”]

Pros/Cons:

- Split long or rare words into smaller, semantically meaningful components or subwords
- No out-of-vocabulary words – any non-subword token can be constructed from other subwords (always includ all characters as subwords)
- Examples algorithms for learning a subword tokenization:
 - Byte-Pair-Encoding (BPE), WordPiece, SentencePiece

GREEDY DECODING FOR A LANGUAGE MODEL

Background: Greedy Search



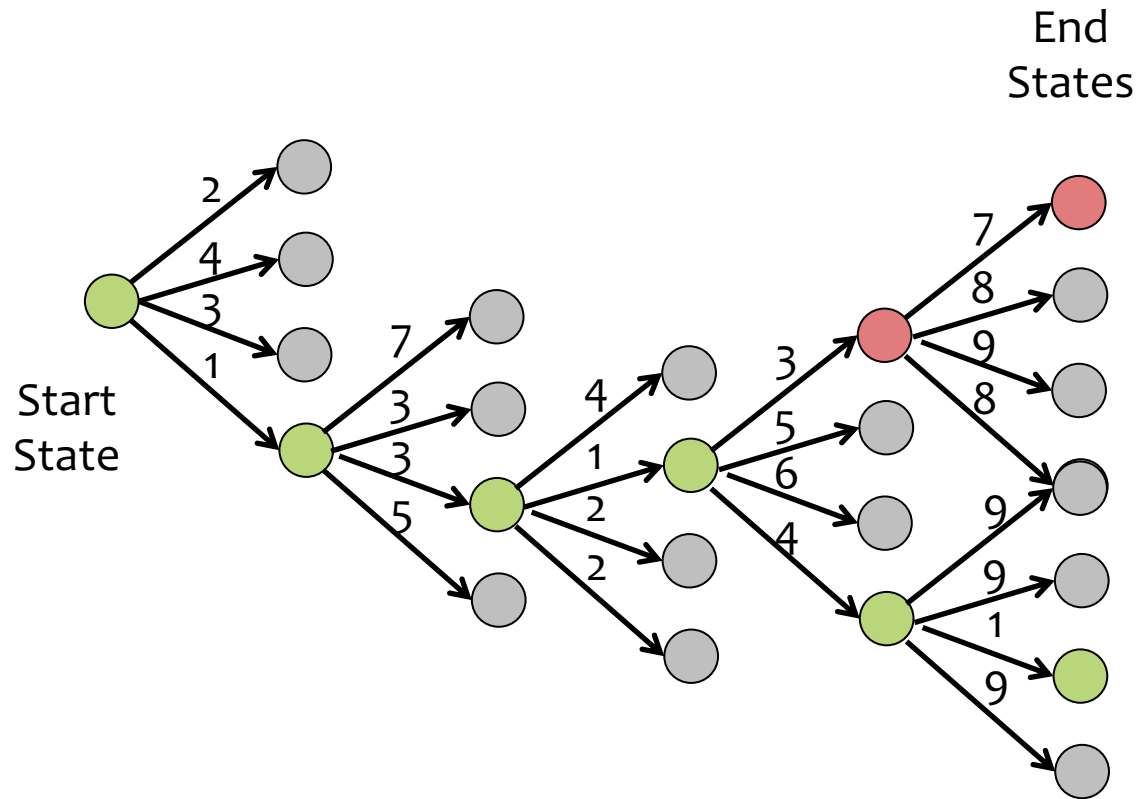
Goal:

- Search space consists of nodes and weighted edges
- Goal is to find the lowest (total) weight path from root to a leaf

Greedy Search:

- At each node, selects the edge with lowest (immediate) weight
- **Heuristic** method of search (i.e. does not necessarily find the best path)
- Computation time: **linear** in max path length

Background: Greedy Search



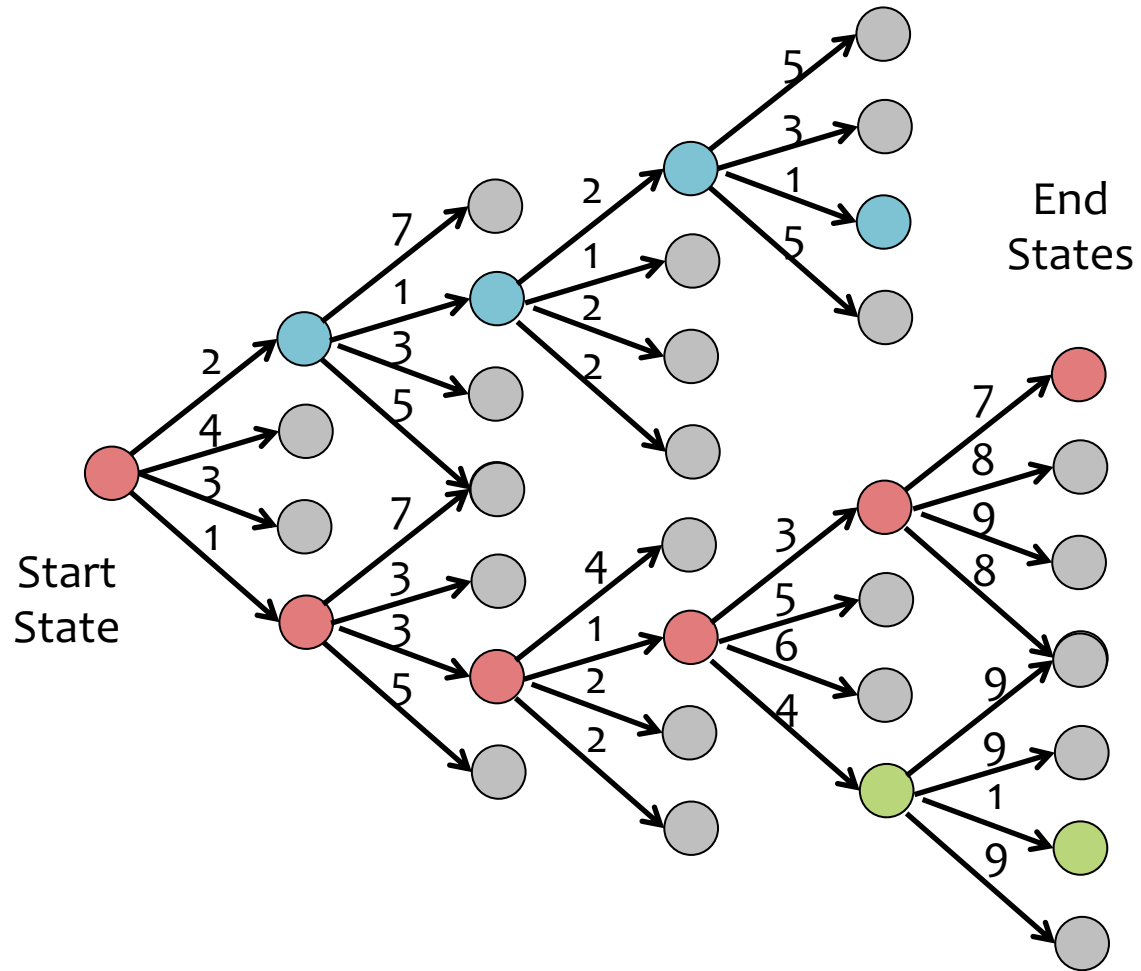
Goal:

- Search space consists of nodes and weighted edges
- Goal is to find the lowest (total) weight path from root to a leaf

Greedy Search:

- At each node, selects the edge with lowest (immediate) weight
- **Heuristic** method of search (i.e. does not necessarily find the best path)
- Computation time: **linear** in max path length

Background: Greedy Search



Goal:

- Search space consists of nodes and weighted edges
- Goal is to find the lowest (total) weight path from root to a leaf

Greedy Search:

- At each node, selects the edge with lowest (immediate) weight
- **Heuristic** method of search (i.e. does not necessarily find the best path)
- Computation time: **linear** in max path length

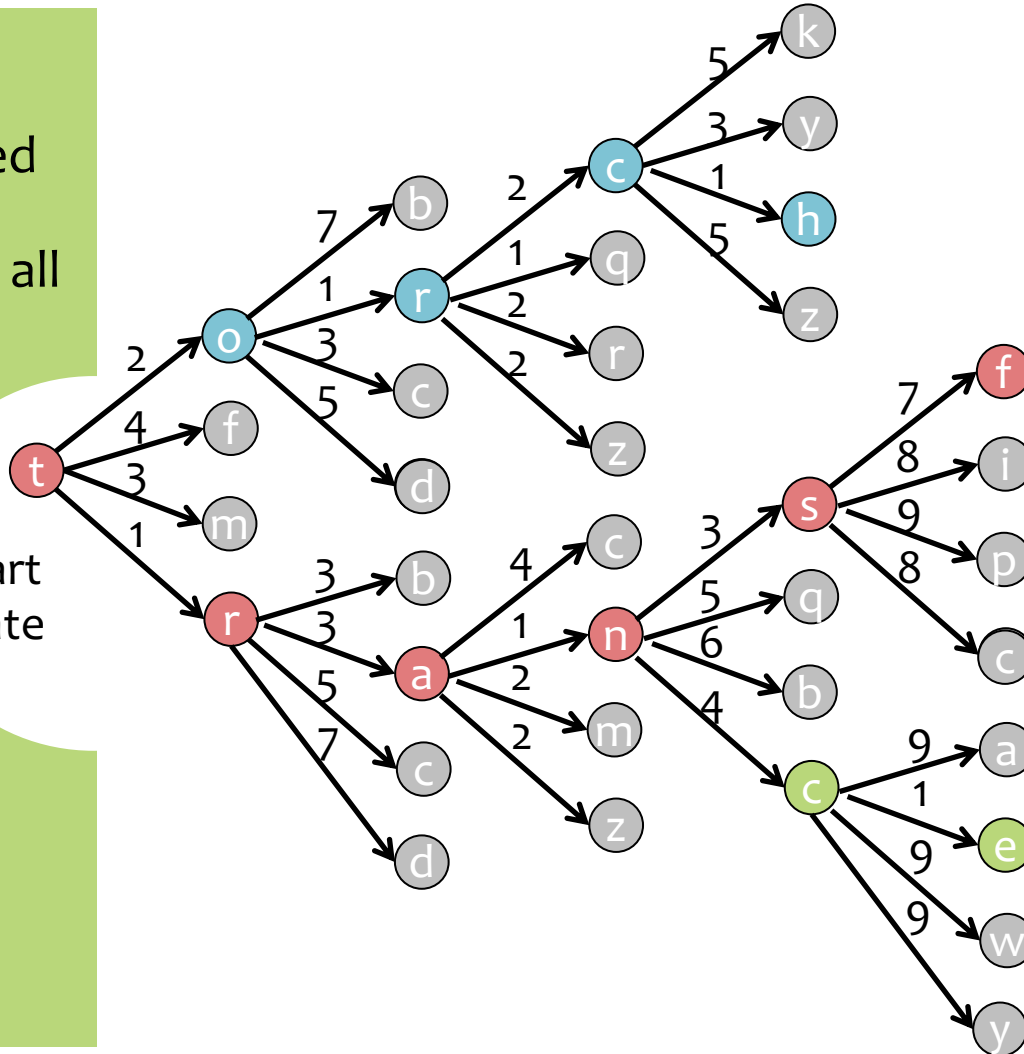
Greedy Decoding for a Language Model

Setup:

- Assume a character-based tokenizer
- Each node has all characters {a,b,c,...,z} as neighbors

Start State

- Here we only show the high probability neighbors for space



Goal:

- Search space consists of nodes (partial sentences) and weighted by negative log probability
- Goal is to find the highest probably (lowest negative log probability) path from root to a leaf

Greedy Search:

- At each node, selects the edge with lowest negative log probability
- **Heuristic** method of search (i.e. does not necessarily find the best path)
- Computation time: **linear** in max path length

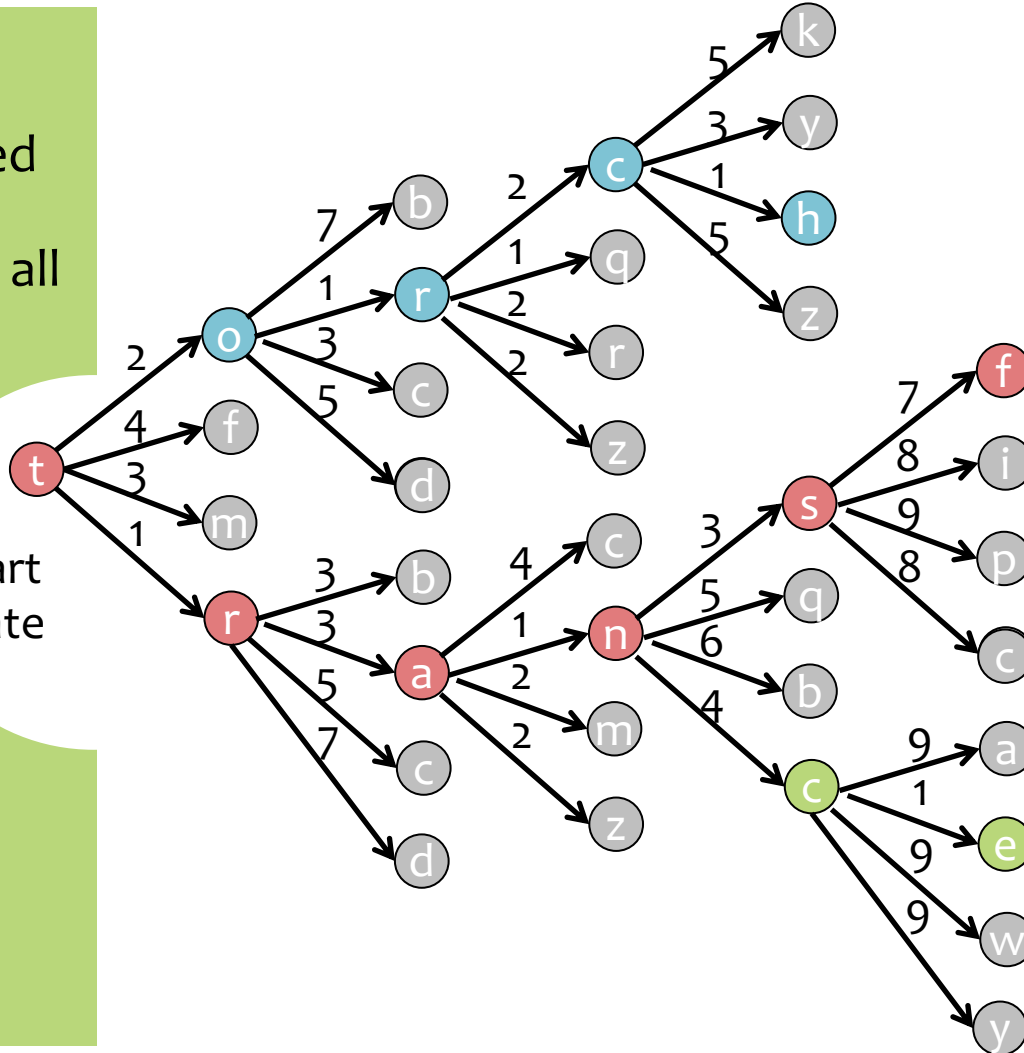
Sampling from a Language Model

Setup:

- Assume a character-based tokenizer
- Each node has all characters {a,b,c,...,z} as neighbors

Start State

- Here we only show the high probability neighbors for space



Goal:

- Search space consists of nodes (partial sentences) and weighted by negative log probability
- Goal is to sample a path from root to a leaf with probability according to the probability of that path

Ancestral Sampling:

- At each node, randomly pick an edge with probability (converting from negative log probability)
- **Exact** method of sampling, assuming a locally normalized distribution (i.e. samples a path according to its total probability)
- Computation time: **linear** in max path length